Project : Secure File Storage and Access Management for Project Teams

Name : Mohammad Aamir Siddique

Date : 03/03/2026

1. Project Objective

To develop a secure file storage and access system for finance teams at Globex Financial, focusing on user access controls, Access Control List (ACL) permissions, command history management, and security violation reporting.

2. Project Description

This project aims to enhance file security and access management, prevent unauthorized modifications, and ensure compliance with audit requirements. It focuses on role-based access control (RBAC), robust command history tracking, and real-time security monitoring to actively mitigate insider risks and unauthorized access.

3. Real-Time Scenario Context

Globex Financial recently faced a security breach where an unauthorized user accessed and modified confidential financial reports, exposing weak access controls and monitoring gaps within the organization. To enhance security, the company is implementing a secure access management system with strict permissions and real-time monitoring.

This system enforces user-specific access controls via ACLs, allowing only file creators to modify or delete files, while restricting others to read-only access. It tracks command history with precise timestamps for auditing and logs unauthorized access attempts for IT review. Finally, a secure web dashboard enables real-time security monitoring, ensuring continuous protection and regulatory compliance.

4. Industry Tools Utilized

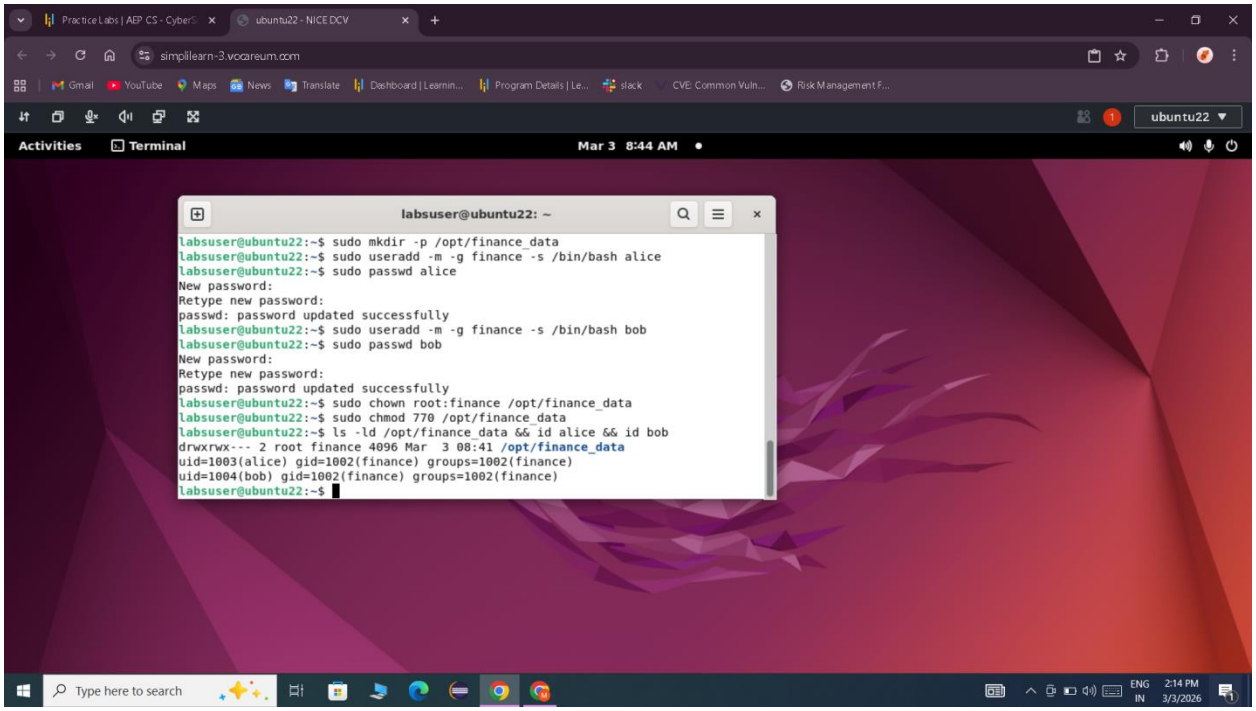
1. **Linux Command-Line & ACLs:** Used in enterprise IT to enforce Role-Based Access Control (RBAC) and restrict unauthorized file modifications at a granular level.
2. **Shell Configuration (.bashrc):** Helps automate security policies and enforce user-specific environment settings, such as timestamped audit trails.
3. **Syslog/Rsyslog:** Essential for capturing and routing system activities and security events, aiding in compliance and forensic analysis.
4. **HTML & PHP (Web Reporting Interface):** Enables real-time security monitoring through customized web-based dashboards for IT security operations centers (SOC).

---

5. Execution Methodology & Logs

Phase 1: User Provisioning & Directory Setup

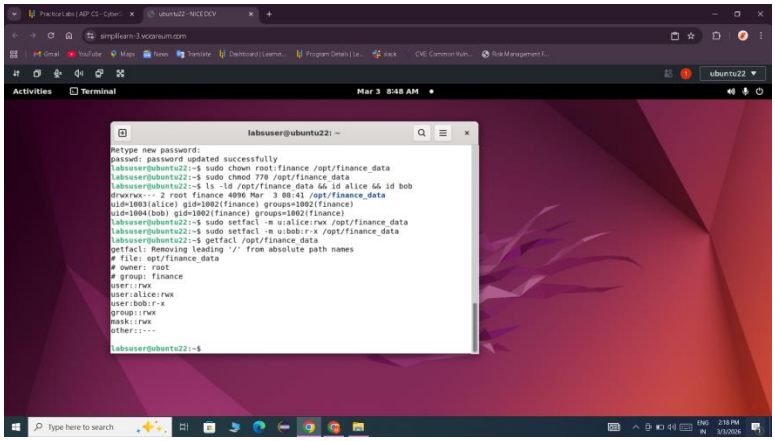
Description: To establish a secure foundation, a dedicated `finance` user group was created. Two distinct employee accounts (`alice` and `bob`) were provisioned and assigned to this group. A secure directory (`/opt/finance_data`) was generated, its ownership was strictly assigned to the `finance` group, and baseline permissions were locked down (770) to prevent unauthorized outside access.



Terminal output confirming the successful creation of the finance group, the provisioned user accounts, and the baseline 770 directory permissions.

Phase 2: Implementing Access Control Lists (ACLs)

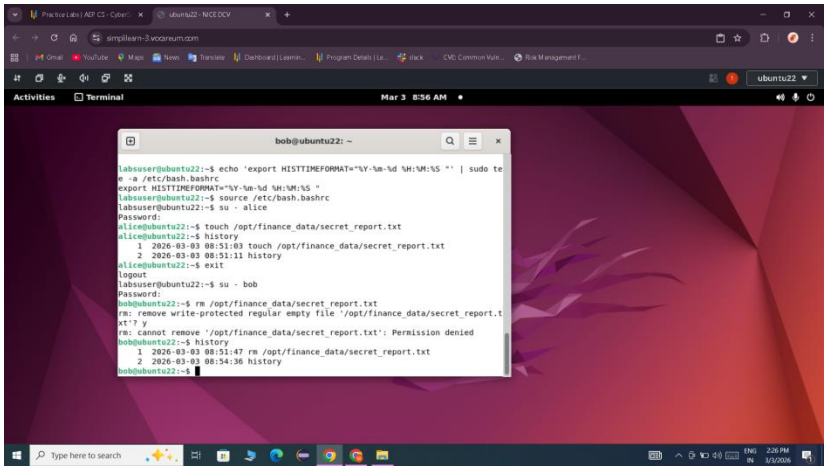
Description: Standard group permissions were insufficient for the scenario's strict requirements, so precise ACLs were deployed. User `alice` (simulating a creator) was granted explicit Read/Write/Execute (`rwx`) permissions, while user `bob` (simulating a viewer) was strictly limited to Read/Execute (`r-x`), stripping his ability to modify or delete files.



Output of the `getfacl` command verifying that granular, user-specific access rules have been successfully applied to the finance_data directory.

Phase 3: Command Auditing & Permission Verification

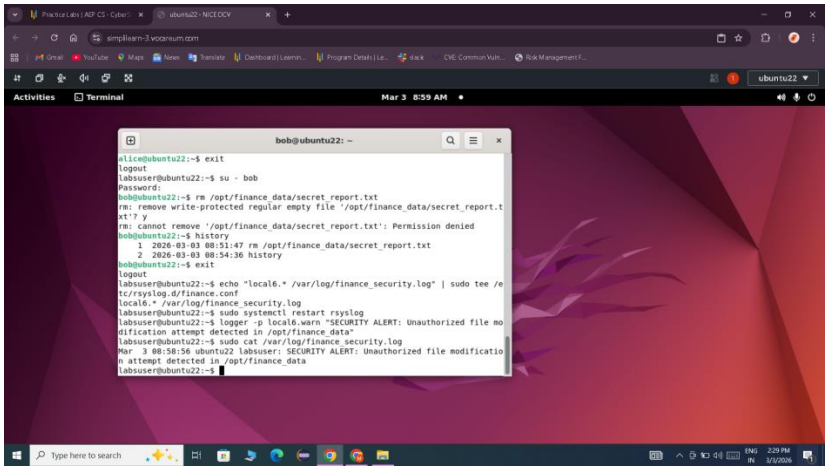
Description: To meet compliance requirements, the global `.bashrc` profile was modified to inject exact `HISTTIMEFORMAT` timestamps into all user command logs. To test the security posture, Alice successfully created a classified file. Bob then attempted to delete it. The system actively blocked Bob's unauthorized attempt, and the timestamped history log captured the violation for forensic review.



Forensic proof showing the system blocking Bob's unauthorized deletion attempt ("Permission denied") and capturing the exact timestamp of the violation in the history log.

Phase 4: Security Violation Reporting (Rsyslog)

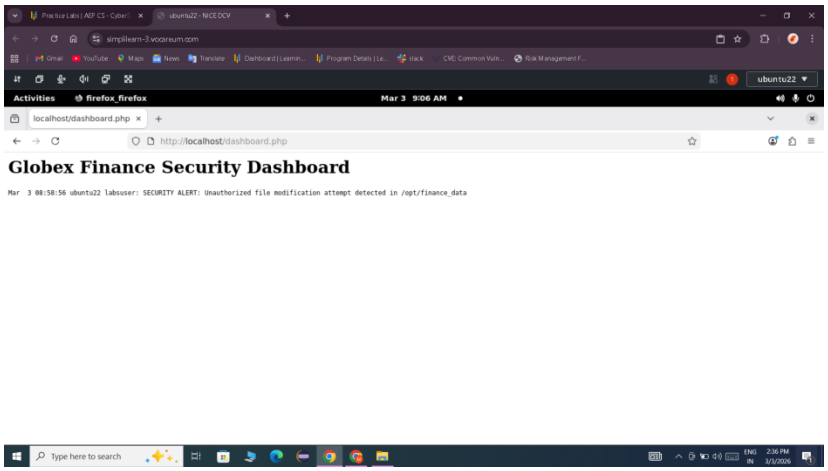
Description: To elevate local errors into enterprise-level alerts, the Linux `rsyslog` daemon was configured. A custom rule (`finance.conf`) was deployed to route specific local security facilities to a dedicated `/var/log/finance_security.log` file. A simulated intrusion alert was then successfully injected and verified via the command line.



Terminal confirmation showing the custom rsyslog configuration successfully captured and routed the simulated "SECURITY ALERT" warning.

Phase 5: Web Dashboard Integration

Description: To allow IT staff to monitor threats without terminal access, an Apache web server with PHP support was deployed. The permissions on the `finance_security.log` file were adjusted to allow the web daemon to read it. Finally, a custom `dashboard.php` file was coded to pull the live log file directly into a localized web interface.



Mozilla Firefox successfully rendering the Globex Finance Security Dashboard, displaying the live Rsyslog alert stream.

6. Final Project Conclusion

This project successfully established a highly secure, heavily monitored file storage environment for the Globex Financial team. By layering baseline Linux directory permissions with strict Access Control Lists (ACLs), the system guarantees that only authorized file owners can modify data, directly neutralizing insider threats. Furthermore, the integration of timestamped bash histories, custom Rsyslog alerting, and a live PHP web dashboard ensures that the IT Security team maintains complete, real-time visibility over all access attempts, fully satisfying the scenario's compliance and auditing requirements.